



**PROJECT BASED LEARNING (PBL-2) LAB
(CSP297)**

Elasticity Efficiency in Cloud Storage using Auto-Scaling Groups

B.TECH 2nd YEAR

SEMESTER: 4th

SESSION: 2024-2025

Submitted By:

Swagata Sinha (2024104171)

SECTION: CSK

Submitted To

**Dr. Avinash Kumar
(Assistant Professor)**

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
SHARDA SCHOOL OF COMPUTING SCIENCE AND ENGINEERING
SHARDA UNIVERSITY, GREATER NOIDA**

Table of Contents

Page No

Project Title.....	3
Team / Group Formation:	3
Technologies to be used.....	3
Tools	3
Problem Statement	4
Literature Survey	4
Project Description.....	5
Project Modules: Design/Algorithm	5
Implementation Methodology	6
Result & Conclusion.....	8
Future Scope and further enhancement of the Project	9
Advantages of this Project	9
Outcome.....	10
References.....	10

Project Title

Elasticity Efficiency in cloud storage using Auto-Scaling Groups

Team / Group Formation:

S. No	Student Name	Roll Number	System ID	Role
1	Swagata Sinha	2401010629	2024104171	Design, Developer, Researcher

Technologies to be used

1) Software Platform:

- Cloud Platform: AWS (Amazon Web Services)
- Services used: EC2, Auto-Scaling Group (ASG), CloudWatch, EBS, AMI, AWS Cost Calculator

2) Front-end:

- AWS Management Console (No UI coding required)

Tools

Tool Name	Purpose
AWS EC2 (Elastic Compute Cloud)	Launch and manage compute instances for workload execution
AWS EBS (Elastic Block Store)	Provides persistent storage volumes attached to EC2 instances
AWS Auto Scaling Group	Manages elasticity by automatically scaling instances up or down
AWS CloudWatch	Collects CPU and network metrics and triggers alarms for scaling actions
AWS Application Load Balancer	Ensures even traffic distribution across multiple instances
AWS Cost Calculator	Estimates operational costs for static vs dynamic provisioning

Stress Workload Generators	<i>Stimulates CPU and network load to mimic storage-intensive applications</i>
SSH (Secure Shell)	<i>Enables remote access to EC2 instances for executing stress commands</i>

Problem Statement

Existing auto-scaling approaches are limited in several ways:

- 1) Dynamic workloads: Cloud storage workloads are highly unpredictable with fluctuating read/write demands.*
- 2) Single-metric limitation: Traditional auto-scaling relies mainly on CPU utilization, ignoring network traffic essential for storage-intensive applications*
- 3) Most existing studies are theoretical or stimulation-based, lacking real-world AWS implementation*
- 4) Cost-awareness gap: Elasticity is often evaluated only in terms of performance, with limited focus on operational cost trade-offs.*
- 5) Self-healing underexplored: Auto-Scaling Groups provide fault-tolerance, but their self-healing capabilities are rarely evaluated alongside elasticity efficiency*

Absence of Framework: Lack of a practical, multi-metric, cost-aware evaluation framework for measuring elasticity efficiency in storage -intensive workloads using AWS Auto-Scaling Groups

Literature Survey

Font: Times New Roman; Font Size: 12, Color Black

<i>S.No.</i>	<i>Author(s)</i>	<i>Contribution</i>	<i>Limitation</i>
<i>1.</i>	<i>Lorido - Botrán et al.</i>	<i>Classified auto-scaling strategies into reactive and proactive methods; emphasized cost-performance trade-offs</i>	<i>Primarily focused on CPU-based scaling; lacks multi-metric implementation</i>
<i>2.</i>	<i>Chen et al.</i>	<i>Introduced feedback loop (Monitor-Analyze-Act) and multi-objective optimization; Proposed taxonomy for self-adaptive cloud systems</i>	<i>Limited practical implementation in real-cloud environments</i>

3.	<i>Patel et al.</i>	<i>Highlighted importance of cost-aware frameworks in auto-scaling systems</i>	<i>Lacks integration with real-time performance metrics</i>
4.	<i>Kaur et al.</i>	<i>Demonstrated need for multi-metric scaling for storage-intensive workloads beyond CPU utilization</i>	<i>Does not provide full experimental validation on cloud platforms</i>
5.	<i>Herbst et al.</i>	<i>Introduced elasticity benchmarking techniques for evaluating cloud performance</i>	<i>Lacks cost-aware efficiency indicators and real-world applicability</i>

Project Description

The project focuses on evaluating Elasticity Efficiency in Cloud Storage workloads using Auto-Scaling Groups (ASGs). A comparative study is conducted between static provisioning (fixed EC2 + EBS) and dynamic provisioning (ASG based scaling). Synthetic CPU and network stress to mimic storage-intensive applications. CloudWatch collects CPU utilization and network traffic metrics. Multi-Metric scaling policies are implemented to overcome CPU- only limitation. This project demonstrates self-healing behavior of ASGs where it automatically replaces unhealthy instances to ensure resilience. Also, an empirical lightweight indicator Elasticity Efficiency Index (EEI) is introduced to quantify efficiency relative to cost. Outcomes highlight improved cost-performance, reduced idle resource wastage, and enhanced resilience and aligns with SDG 12 by promoting responsible cloud resource consumption.

Project Modules: Design/Algorithm

The system design integrates monitoring, scaling, self-healing, and evaluation layers to ensure elasticity efficiency in cloud storage workloads. The logical flow is based on AWS components and follows a feedback-driven adaptive model.

1) End Users:

- *Generate dynamic read/write requests for the cloud storage application*
- *Requests are routed through the Application Load Balancer (ALB)*

2) Application Load Balancer (ALB):

- *Distributes incoming traffic evenly across EC2 instances*
- *Ensures high availability and prevents overload on a single resource*

3) Auto Scaling Group (ASG):

- Manages multiple EC2 instances with attached Elastic Block Store (EBS)
- Dynamically adjusts the number of instances based on workload demand

4) CloudWatch Monitoring:

- Collects real-time metrics: CPU Utilization and Network In traffic
- Provides input for scaling decisions

5) Scaling and Control Layer:

- Implements multi-metric scaling policies (CPU + Network)
- Uses CloudWatch alarms to trigger scale-out or scale-in actions

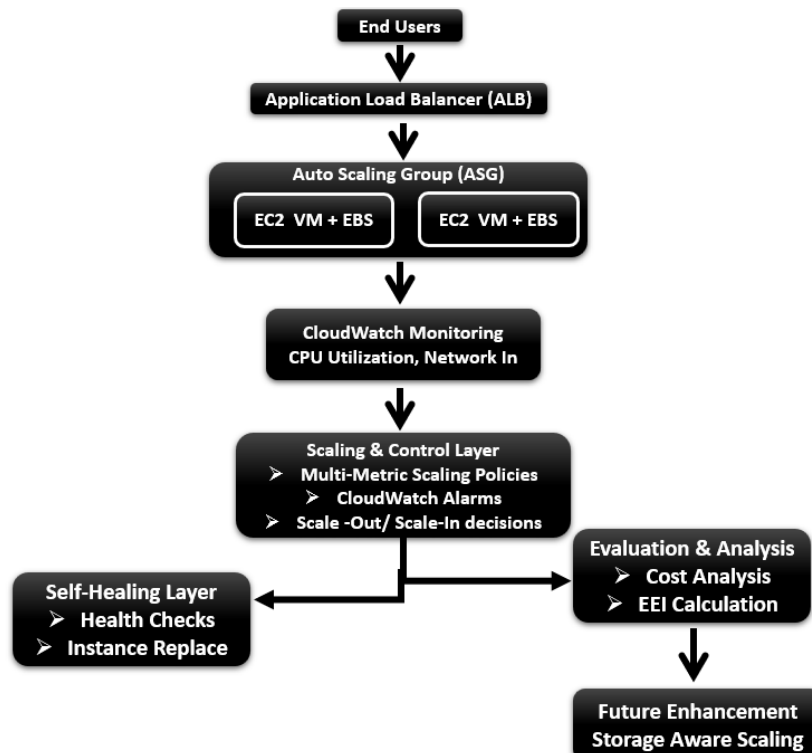
6) Self-Healing Layer:

- Performs health checks on EC2 instances
- Automatically replaces unhealthy or terminated instances to maintain resilience

7) Evaluation and Analysis Layer:

- Conducts cost-aware elasticity evaluation
- Computes the Elasticity Efficiency Index (EEI) to quantify efficiency relative to cost

Implementation Methodology



The project adopts an infrastructure-level experimental methodology to evaluate elasticity efficiency in cloud storage environments using AWS Auto Scaling Groups (ASGs). The methodology is structured into phases to ensure reproducibility and alignment with research practices.

A. Experimental Setup:

- *Static Environment: A single EC2 instance with attached Elastic Block Store (EBS), representing fixed resource provisioning.*
- *Dynamic Environment: An Auto Scaling Group (ASG) with multiple EC2 instances created from a common AMI, representing elastic provisioning.*
- *Both environments were deployed in the same AWS region with identical configurations to ensure fairness.*

B. Workload Generation:

- *Synthetic workloads were generated using stress tools*
- *CPU-intensive operations stimulated compute demand*
- *Network traffic stimulated storage access behavior*
- *This avoided application-specific complexity while ensuring reproducibility*

C. Monitoring and Metrics Collection:

- *CloudWatch was used for continuous monitoring*
- *Metrics collected: CPU Utilization (compute demand) and Network In traffic (storage access behavior)*
- *Data was recorded for both static and dynamic environments during workload execution*

D. Auto-Scaling Policy Configuration:

- *Threshold-based scaling policies were adopted for simplicity and reproducibility*
- *CPU Policy: Scale-out when traffic exceeded threshold*
- *CloudWatch alarms triggered event-driven scaling actions automatically*

E. Self-Healing Mechanism:

- *ASG's built-in health checks were utilized to demonstrate resilience*
- *Manual termination of an EC2 instance triggered automatic replacement*

- *This validated fault tolerance and service continuity*

F. Evaluation and Analysis:

- *Comparative evaluation was performed based on:*
 - *Resource utilization patterns*
 - *Instance scaling behavior*
 - *Estimated operational cost*
- *A lightweight Elasticity Efficiency Index (EEI) was computed:*

$$EEI = \frac{CPU + Network}{Cost}$$

- *EEI served as an empirical indicator for comparing static vs dynamic provisioning*

Result & Conclusion

<Guidelines: *The write-up must end with the concluding remarks-briefly describing innovations in the*

- *Static Environment:*
 - *Fixed instance count led to high CPU spikes during load*
 - *Significant resource wastage during idle periods*
 - *No fault tolerance; single point of failure*
 - *Constant operational cost regardless of demand*
- *ASG-based Environment:*
 - *Dynamic instance scaling improved load distribution*
 - *Idle resource usage reduced significantly*
 - *Automatic failure recovery through self-healing*
 - *Better cost-performance balance achieved*
- *Quantitative Outcomes:*
 - *EEI (Static) $\approx$ 2947.9*
 - *EEI (Dynamic) $\approx$ 3962.9*

- *Efficiency gain $\approx 34.4\%$*
- *Demonstrated that ASG-based elasticity improves efficiency, resilience, and cost-awareness to static provisioning*

Conclusion:

The project successfully demonstrates that AWS Auto-Scaling Groups provide superior elasticity efficiency for storage-intensive workloads. By integrating multi-metric scaling (CPU + Network), self-healing, and cost-aware evaluation, the system achieves improved resilience, reduced wastage and sustainable cloud operations.

Future Scope and further enhancement of the Project

<Guidelines: Explain how this project can be enhanced in terms of functions, usage etc.>

- *Storage-Aware Scaling: Extend elasticity models to include storage-specific metrics (e.g., IOPS, disk throughput)*
- *Predictive Scaling: Use machine learning models to forecast workload demand and proactively adjust resources.*
- *Multi-Cloud Comparative Analysis: Evaluate elasticity efficiency across AWS, Azure, and GCP for broader applicability*
- *Advanced Cost-Optimization: Integrate spot instances and reserved pricing strategies for deeper cost savings*
- *Application-Aware Elasticity: Tailor Scaling policies to specific workload types (e.g., database-heavy vs compute-heavy)*
- *Enhanced Visualization: Develop dashboards for real-time elasticity monitoring and EEI tracking.*

Advantages of this Project

Font: Times New Roman; Font Size: 12, Color Black

- *Students and Researchers: Gain a practical, reproducible framework for studying elasticity efficiency in real-cloud environments*
- *Cloud Developers: Benefit from multi-metric scaling (CPU + Network) strategies that improve application performance and reduce idle costs*
- *Organizations: Achieve cost-aware resource management, minimizing over-provisioning and operational waste*

- *System Administrators: Leverage ASG's self-healing mechanism to ensure resilience and continuous availability*
- *Sustainability Advocates: Support SDG 12 by promoting responsible consumption of cloud resources and reducing energy waste.*

Outcome

The research paper on elasticity efficiency in cloud storage using AWS Auto Scaling Groups has been successfully prepared and is in the process of submission to a Scopus-indexed journal. The outcome of this project is research-oriented, contributing to academic literature in cloud computing and providing a foundation for future innovations in storage-aware scaling and elasticity efficiency.

References

<Guidelines: Give references, Example: Books referred, website URL, any other >

1. T. Lorigo-Botran, J. Miguel-Alonso, and J. A. Lozano, "A review of auto-scaling techniques for elastic applications in cloud environments," *Journal of Grid Computing*, vol. 12, no. 4, pp. 559–592, 2014. doi: 10.1007/s10723-014-9314-7.
2. J. O. Kephart and D. M. Chess, "The vision of autonomic computing," in *Computer*, vol. 36, no. 1, pp. 41–50, Jan. 2003, doi: 10.1109/MC.2003.1160055.
3. P. Patel, A. Ranabahu, and A. Sheth, "Service Level Agreement in Cloud Computing," *Cloud Workshops Proceedings*, pp. 1–10, 2009.
4. Zhenhuan Gong, Xiaohui Gu and J. Wilkes, "PRESS: PRedictive Elastic ReSource Scaling for cloud systems," 2010 International Conference on Network and Service Management, Niagara Falls, ON, Canada, 2010, pp. 9–16, doi: 10.1109/CNSM.2010.5691343. keywords: {Presses;Prediction algorithms;Resource management;Time series analysis;Markov processes;Predictive models;Measurement},
5. N. Roy, A. Dubey and A. Gokhale, "Efficient Autoscaling in the Cloud Using Predictive Models for Workload Forecasting," 2011 IEEE 4th International Conference on Cloud Computing, Washington, DC, USA, 2011, pp. 500–507, doi: 10.1109/CLOUD.2011.42. keywords: {Time factors;Resource management;Prediction algorithms;Quality of service;Virtual machining;Throughput;Cost function;autoscaling;workload forecasting;predictive models;cloud computing;performance models},
6. N. Herbst, S. Kounev, and R. Reussner, "Elasticity in Cloud Computing: What It Is, and What It Is Not," *Proceedings of the 10th International Conference on Autonomic Computing (ICAC)*, pp. 23–27, 2013.
7. Amazon Web Services, "Auto Scaling Groups Documentation," AWS, 2025.

8. Amazon Web Services, "CloudWatch Metrics and Monitoring," AWS, 2025.

9. M. Mao and M. Humphrey, "A Performance Study on the VM Startup Time in the Cloud," 2012 IEEE Fifth International Conference on Cloud Computing, Honolulu, HI, USA, 2012, pp. 423-430, doi: 10.1109/CLOUD.2012.103. keywords: {Linux;Availability;Time measurement;Servers;Asia;Cloud computing;Elasticity;cloud computing;VM startup/acquisition/spinup time;performance study;spot instances},

Signature

Student Name	Student Sign	Faculty Name	Faculty Sign